tools or algorithms are not designed to handle large data sets, resulting in failures when applied to such data.
- *Out of memory issues:* The heap memory or allocated memory (RAM) may not be sufficient to handle large data sets.
- *Programs might run infinitely:* The processing time required to handle large data might grow exponentially.
- *Complex transformation requires redesign and more development effort:* Data processing teams often need to re-design and re-engineer their data pipelines, leading to higher development efforts and project timeline impacts.

Techniques for handling large data sets

To address the common challenges that we just discussed, several techniques can be employed.

1. Allocate more memory to meet the higher memory requirements.
2. Choose instances with high memory/CPU ratio when the volume is high, and the transformations/computations are low.
3. Avoid unnecessary and complex joins wherever possible.
4. Divide the data set into smaller chunks and process them.
5. Choose a data format that is optimised for memory utilisation.
6. Stream data or process in (near) real-time so that data processing happens when the data is available rather than storing and processing in large batches.
7. Progressively load data into memory by using appropriate timer techniques.
8. Use an appropriate Big Data platform that meets the needs of the business.
9. Use a suitable programming language and technology to cater to the data processing needs.
10. Leverage distributed processing patterns.
11. Use parallel computing architecture.
12. Build a workflow and orchestration platform.
13. Use in-memory data sets for small data sets.
14. Leverage in-built operators for efficient processing.
15. Use appropriate partitioning techniques.
16. Use indexing for fetching as much data as required.
17. Leverage garbage collection techniques.
18. Flush data from memory and cache for efficiency.
19. Create dynamic clusters based on input size patterns.
20. Use lazy loading techniques.

A few technical tips

The data engineering team needs to have a holistic approach to handle large data sets. Some of the technical tips for this include:

1. Specify the correct and appropriate data types while importing data. For example, in Pandas, we can use uint32 for positive integers.
2. Store the files (data sets) in zipped format such as the *tar.gz* file format.
3. If the processing requirements are for data warehousing and reporting, use columnar NoSQL databases like Cassandra.
4. Persist intermittent data sets to avoid re-computing the entire pipeline in case of failures. For example, use the *df.write()* method.
5. Use appropriate system methods to understand the memory availability and dynamically handle the large data sets in various chunks (partitions). Examples of the methods are:
   a. *df.memory_usage()*
   b. *gc.collect()*
   c. *sys.getsizeof()*
6. For storing transactional data, use PostgreSQL or MySQL. They also allow clustering techniques and replication capabilities which can be leveraged appropriately.
7. Always read from external sources in appropriate fetch sizes and write in appropriate batch sizes. For example, in Spark, one can use *fetchsize()* and *batchsize()* options.
8. Create a database collection pool; for example, setMinPoolSize and setMaxPoolSize.

Handling large data sets requires a holistic approach. Data engineers need to carefully consider the entire data pipeline, including the frequency and size of data input. They should include appropriate data quality checks in the pipeline to ensure the sanctity of the data. Also, a proper error handling mechanism, graceful shutdown techniques, and retry mechanisms in data pipelines are essential. END

By: Phani Kiran

The author has over 19 years of experience in the IT industry and has worked on various digital transformation initiatives.